

第28回 実践：Web ゲームプログラミング7（ゲームオーバーとリスタート）

実習時のフォルダ構成

Web_名前 の中に 28 フォルダを作る。第 27 回の `sample` フォルダと `exercise` フォルダを、28 の中にコピーして、続きから作る。

```
Web_名前
├── 28
│   ├── sample          ← 第 27 回の sample をコピーする
│   │   ├── index.html
│   │   ├── style.css
│   │   └── main.js
│   └── exercise        ← 第 27 回の exercise をコピーする
│       ├── index.html
│       ├── style.css
│       └── main.js
```

- `sample`：講義で使うサンプル（講義内容のコードは、`sample/main.js` に書き足す）
- `exercise`：自分のシューティングゲーム（演習 1～2、発展）

講義内容

`sample/main.js` に書き足す。書いたら `sample/index.html` をブラウザで開いて確認する。

1. 前回の復習

- `checkPlayerHit`：敵が自機にぶつかったら、敵を消して、残機（`lives`）を 1 減らす。
- `drawUI`：`fillText` とテンプレートリテラルで、得点と残機を表示する。

今回は、残機が 0 になったらゲームオーバーにし、`Enter` キーでリスタートできるようにする。これで、シューティングゲームの基本が完成する。

2. 今回の `main.js` の並び

```
// ===== 準備 =====
// ===== ゲームの状態 ===== ← gameState を追加する
// ===== 自機・弾・敵 =====
// ===== キー入力 ===== ← Enter キーを追加する
// ===== リスタート ===== ← 今回追加する (resetGame)
// ===== 更新 ===== ← checkPlayerHit を書き換える
// ===== 描画 ===== ← drawGameOver を追加し、draw を書き換える
// ===== ゲームループ ===== ← gameLoop を書き換える
```

3. ゲームの状態 (gameState)

ゲームには、「プレイ中」「ゲームオーバー」のような「状態」がある。状態によって、行う処理が変わる。

状態	gameState の値	更新（動かす）	描画
プレイ中	"play"	する	自機・弾・敵・得点など
ゲームオーバー	"gameover"	しない（止まる）	上と同じもの + 「GAME OVER」の文字

今の状態を、文字列の変数 `gameState` に入れておく。「ゲームの状態」の部分の先頭に追加する。

```
// ===== ゲームの状態 =====
let gameState = "play";           // "play": プレイ中  "gameover": ゲームオーバー
let score = 0;
let lives = 3;
let frameCount = 0;              // ゲーム開始からのフレーム数
```

4. 残機 0 でゲームオーバーにする

「更新」の部分の `checkPlayerHit` を、次のように書き換える (`if (lives <= 0)` の 3 行を追加する)。

```
// 敵と自機の当たり判定
function checkPlayerHit() {
  for (let i = enemies.length - 1; i >= 0; i--) {
    if (isHit(player, enemies[i])) {
      enemies.splice(i, 1);
      lives -= 1;
      if (lives <= 0) {
        gameState = "gameover";
      }
      break;           // 1 フレームで減る残機は 1 つまで
    }
  }
}
```

5. プレイ中だけ更新する

「ゲームループ」の部分の `gameLoop` を、次のように書き換える。

```
// ===== ゲームループ =====
function gameLoop() {
  if (gameState === "play") {
    update();
  }
  draw();
  requestAnimationFrame(gameLoop);
}
```

- `gameState` が `"play"` のときだけ、`update` を呼び出す。ゲームオーバーになると、自機も弾も敵も止まる。

- `draw` と `requestAnimationFrame` は、状態に関係なく続ける。ゲームループは止めない。

6. ゲームオーバーの画面を描く

「描画」の部分の `drawUI` の下に `drawGameOver` を追加し、`draw` を書き換える。

```
function drawGameOver() {
  ctx.fillStyle = "#ffffff";
  ctx.textAlign = "center";
  ctx.font = "40px sans-serif";
  ctx.fillText("GAME OVER", canvas.width / 2, 230);
  ctx.font = "20px sans-serif";
  ctx.fillText("Enter キーでリスタート", canvas.width / 2, 280);
}

function draw() {
  drawBackground();
  drawPlayer();
  drawBullets();
  drawEnemies();
  drawUI();
  if (gameState === "gameover") {
    drawGameOver();
  }
}
```

- ゲームオーバーのときだけ、最後に「GAME OVER」を重ねて描く。
- `textAlign` を `"center"` にして、キャンバスの横の真ん中 (`canvas.width / 2`) に文字を置く。
`drawUI` の最初で毎回 `"left"` に戻しているので、得点の表示はずれない。

7. リスタート (`resetGame`)

リスタートするには、ゲームの途中で変わった変数を、**すべて**最初の値に戻す。「キー入力」の部分の下に、「リスタート」の部分を追加する。

```
// ===== リスタート =====
function resetGame() {
  gameState = "play";
  score = 0;
  lives = 3;
  frameCount = 0;
  player.x = 184;
  bullets = [];
  enemies = [];
  shotTimer = 0;
}
```

- `bullets = []`; `enemies = []` : 空の配列を代入し直して、画面の弾と敵をすべて消す。第 24 回・第 25 回で、`bullets` と `enemies` を `let` で作ったのは、ここで代入し直すため。
- `player` は `const` だが、中の `x` は書き換えられる (第 20 回)。
- 戻し忘れた変数があると、リスタート後に、前のゲームの得点や敵が残ってしまう。

- 最初の値は、ファイルの先頭と `resetGame` の 2 か所を書いてある。値を変えるときは、両方を変える。

8. Enter キーでリスタートする

「キー入力」の部分の `keydown` に、`Enter` キーの処理を追加する。

```
document.addEventListener("keydown", function (event) {
  if (event.key === "ArrowLeft") {
    isLeftPressed = true;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = true;
  }
  if (event.key === " ") {
    isSpacePressed = true;
  }
  if (event.key === "Enter" && gameState === "gameover") {
    resetGame();
  }
});
```

- `Enter` キーが押され、**かつ**ゲームオーバーのときだけ、`resetGame` を呼び出す。
- プレイ中に `Enter` キーを押しても、リスタートしない。
- 第 20 回では「関数は、呼び出すより前に作ることを基本にする」とした。`resetGame` は、この `keydown` の関数より下で作っているが、`keydown` の中の処理が実行されるのは、キーが押されたとき（ファイルを最後まで読み込んだあと）なので、問題なく呼び出せる。ファイルの並びを「キー入力 → リスタート」の順にしておくと、キー入力から呼び出す関数がすぐ下にあって読みやすいため、この順にしている。

演習

`exercise/main.js`（自分のシューティングゲーム）に書く。

演習 1 ゲームオーバーとリスタート

- 講義と同じように、残機が 0 になったらゲームオーバーになり、`Enter` キーでリスタートできるようにする。
- `resetGame` では、自分のゲームの最初の値（残機の数、自機の x 座標など）に戻す。
- 次の 2 つを確認する。これで、最終必須仕様（第 30 回）の 9 項目がすべてそろろう。
- 残機の数だけ敵にぶつかると、動きが止まり、「GAME OVER」が表示される。
- `Enter` キーを押すと、得点・残機が最初の値に戻って、新しいゲームが始まる。弾と敵も消えている。

演習2 resetGame を確かめる

- `resetGame` の中の `enemies = [];` の行の先頭に `//` を付けて、ゲームオーバー → リスタートしてみる。何が起きるかを確かめる。
- 確かめたら、`//` を消して元に戻す。

発展（任意）

- ゲームオーバーの画面に、最終得点（「SCORE: ○○」）も表示する。
- ゲームオーバーのとき、画面全体を半透明の黒で塗ってから、文字を描く。（ヒント：`ctx.fillStyle = "rgba(0, 0, 0, 0.6)";` のように書くと、半透明の色を指定できる。`rgba(赤, 緑, 青, 透明度)` の形で、赤・緑・青は 0～255、透明度は 0（透明）～ 1（不透明）の数で指定する。）